

Packages, Encapsulation, and Interfaces

Lecturer : RCH

Teaching Assistant: MEP



INSTITUT TEKNOLOGI DEL

Jl. Sisingamangaraja, Sitoluama Laguboti

Kab. Tobasa SUMUT, Indonesia. 22381

<https://www.del.ac.id/>

Learning Outcomes

1. Package Concept
2. Declaring and Importing Package
3. Encapsulation
4. Access Modifier
5. Interfaces Concept
6. Defining Interfaces
7. Interfaces Example

Package Concept

- Packages are containers for classes that are used to keep the class name spaces compartmentalized.
- It is a mechanism for partitioning the class name spaces into more manageable chunks.
- Packages are mirrored by directories, all the `.class` files must be stored in a directory of your package.
- Name of the directory is name of package.
- Using command prompt: You have to make the folder.
- Using IDE such as Netbeans: it will create the folder for you

Declaring and Importing Package

Declaring Package

The package statement: `package pckg_name;`

Example: `package MyPackage;`

The steps:

1. Put a `package` command as the first statement in a java source file (the file that you want to put on that package).
2. Compile the file. Make sure that the `.class` file is also in the same directory.
3. The package is ready to use.

Declaring and Importing Package

Importing Package

- Java includes the `import` statement to bring certain classes, or entire package, into visibility.
- For example if you already make the package and its classes

```
package mypackage;  
  
public class MyClass {  
    // . . .  
}
```

Declaring and Importing Package

Importing Package

Now, if someone wants to use `MyClass` or, for that matter, any of the other public classes in `mypackage`, they must use the `import` keyword to make the name or names in `mypackage` available:

```
import mypackage.*;  
// . . .  
MyClass m = new MyClass();
```

The alternative is to give the fully qualified name:

```
mypackage.MyClass m = new mypackage.MyClass();
```

Encapsulation

- Encapsulation is a technique of creating a “field” in a class with a “private” attribute and providing access to that “field” via “public methods”.
- If a “field” is declared “private”, that “field” cannot be accessed by anyone outside the “class”, so the field will remain hidden within the “class”.
- Encapsulation is also referred to as “data hiding”.
- Encapsulation can be described as a shield that prevents code and data from being randomly accessed by other code defined outside the "class".

Encapsulation

- Keeping changes to the field object immediately, this can be useful when the new value of a field needs to be validated first
- Hides the internal representation of an object, and only exposes what is relevant from the outside. The internal complexity of the object in question does not need to be exposed elsewhere.
- Keeping the method with public access rights in the class does not need to change when the internal field/method of the class in question changes, so that parts of programs elsewhere that use objects from that class do not need to change as well.
- For ease of debugging, dots in getters and setters are good candidates for debugging breakpoints.

Access Modifier

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Encapsulation

- The encapsulation technique in Java is done by closing direct access to fields belonging to an object from outside the object concerned.

Encapsulation with getters and setters

- **Getters** are methods provided to access the value of a particular field in an object.
- **Setter** is a method provided to replace the value of a specific field in an object.

Getter Setter

```
public class Karyawan {  
    private String nama;  
    private int umur;  
  
    // Getter  
    public String getNama() {  
        return nama;  
    }  
  
    // Setter dengan validasi  
    public void setNama(String nama) {  
        if(nama != null && !nama.isEmpty()) {  
            this.nama = nama;  
        } else {  
            System.out.println("Nama tidak boleh kosong!");  
        }  
    }  
  
    public int getUmur() {  
        return umur;  
    }  
  
    public void setUmur(int umur) {  
        if(umur > 0) {  
            this.umur = umur;  
        } else {  
            System.out.println("Umur harus lebih dari 0!");  
        }  
    }  
}
```

Getter Setter

```
public class Main {  
    public static void main(String[] args) {  
        Karyawan k = new Karyawan();  
        k.setNama("Rudy");  
        k.setUmur(25);  
  
        System.out.println("Nama: " + k.getNama());  
        System.out.println("Umur: " + k.getUmur());  
    }  
}
```

Interfaces Concept

- By using the keyword `interface`, java allows you to fully utilize the “one interface, multiple methods” aspect of polymorphism.
- Have method definitions only
- Have constant variables
- A class can implement **any** number of interfaces
- Classes do not need to be related to use the same interface
- A class must create the complete set of methods defined by the interface

Defining Interfaces

Declaration:

```
access interface name{  
    return-type method-name1(param-list);  
    return-type method-name2(param-list);  
    type final-varname1 = value;  
    type final-varname2 = value;  
    //...  
    return-type method-nameN(param-list);  
    type final-varnameN = value;  
}
```

Defining Interfaces

- The *access* is either public or not used
- When no access specifier → default : only available to the other members of the package.
- Notice that the methods have no bodies. (remember: it looks like prototype in C lang.)
- Variables are implicitly **final** and **static**.
- All methods and variables are implicitly public if the interface is declared as public

Interface Example

```
interface Animal {  
    void makeSound();    // method abstrak  
    void move();         // method abstrak  
}
```

```
class Cat implements Animal {  
    public void makeSound() {  
        System.out.println("Meow... Meow...");  
    }  
    public void move() {  
        System.out.println("Cat walks silently.");  
    }  
}
```

Interface Example

```
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Cat();  
  
        a1.makeSound();  
        a1.move();  
    }  
}
```